

addition'. This process is shown in Figure 2. As the name suggests, this method involves splitting the bits to count; we start by pairing adjacent bits, and summing them. The trick, though, is that we store the intermediate result in the same location as the original number, without destroying the data required in the next step. The code for the procedure is shown below:

```
static inline unsigned char bit_count(unsigned char x)
{
    x = (0x55 & x) + (0x55 & (x >> 1));
    x = (0x33 & x) + (0x33 & (x >> 2));
    x = (0x0f & x) + (0x0f & (x >> 4));
    return x;
}
```

The key points to note for this algorithm are:

- It uses a mask at each step in the algorithm.
- The code takes $O(\log n)$ time to complete.

The masks at each step are shown in Figure 3. The masks in the first step have alternate bits set (0x55); this selects alternate bits for summation. The number above is summed with the same mask (0x55), but with the entire number *shifted right by one bit*. In effect, this sums the alternate bits of the word. The bits that are not important are cleared, and set to 0 in the mask. This procedure is repeated; the goal now is to compute the sum of the intermediate result obtained in the step above. The previous step counted the sum of alternate bits; it is now time to sum *two bits* at a time. The corresponding mask for this step is 0x33 (can you see why?). Again, we repeat the procedure by masking the number with 0x33, and adding to it the result of the number right-shifted by 2 and masked by 0x33. We do something similar in the final step, where we need to count 4 bits at a time, and sum up the result to obtain the final answer.

Figure 2 shows a sample computation for the number 177, which is represented as 10110001.

In Step 1, we sum the adjacent bits, leading to 01100001 (the sum of 1 and 0 is 01, the sum of 1 and 1 is 10 (in binary, this represents 2), the sum of 0 and 0 is 00, the sum of 0 and 1 is 01). In the next step, we sum 2 bits at a time, resulting in 00110001 (the sum of 01 and 10 is 0011 – 3 in binary; the sum of 00 and 01 is 0001). In the final step, we sum 4 bits at a time, resulting in 00000100 (the sum of 0011 and 0001 is 00000100—4 in binary). As expected, this is also the final outcome, and the result of the number of 1s in the block under consideration.

This completes the sideways addition algorithm. As you can see, this algorithm is clearly more efficient than the initial approach.

Exercises

- We focused on 8 bits in a block to explain the algorithm. This algorithm can easily be extended to 32 or 64 bits and beyond. Write a routine to extend this algorithm to 64 bits, and potentially all the way up to 256 bits.
- The algorithm specified above (in the section *Improving the algorithm*) is not necessarily optimal. Look at the references

b7	b6	b5	b4	b3	b2	b1	b0

Figure 1: Typical bit-wise representation

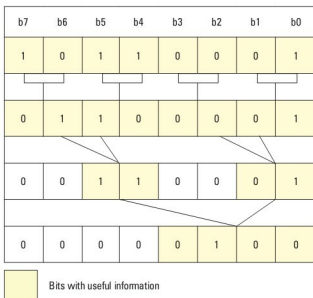


Figure 2: Divide and conquer the summation of bits

b7	b6	b5	b4	b3	b2	b1	b0
0	1	0	1	0	1	0	1
0	0	1	1	0	0	1	1
0	0	0	0	1	1	1	1

Figure 3: Masks to divide and conquer summation

below to see if a more optimal version can be found and used. Explain what optimisations are possible, and how. **END** 

References

- MMIXware: A RISC Computer for the Third Millennium by Donald E Knuth, Springer-Verlag, 1999
- Matters Computational: ideas, algorithms, source code by Jorg Arndt, Draft version of 20-June-2010

By: Balbir Singh

The author has been interested in the kernel and has spent a long time in the industry working on several architectures and kernel programming. A university rank holder, he is also a maintainer of some Linux subsystems.